

## Homework #4: Understanding Image Resizing Using Sampling Theory

March 9, 2026

**Warning:** You are expected to provide answers in your own words based on the discussion held in class. You will receive no credit for copying material from an AI chatbot, the internet, or if your discussion simply does not reflect the class discussion of what is expected. The use of AI or online resources is not prohibited. However, you need to use your own words, and your answers need to align with class discussions.

**Problem #1. Image resizing in Neural Networks.** Image resizing is a required pre-processing step for many Neural Networks. We want to study its effect in the Frequency domain. This problem is studied carefully here.

For the spatial-domain samples, show the continuous-function profiles and a minimum of 25 samples. Use dots for the rest.

For the Frequency-domain sketches, you are required to show 9 tiles centered at the origin. To highlight overlaps, focus on the central tile only. Use dots for the rest.

In PyTorch, the original input is resized using:

<https://docs.pytorch.org/vision/main/generated/torchvision.transforms.Resize.html>.

Inside Neural Networks, image resizing is handled using:

<https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.interpolate.html>.

Direct resizing based on direct sampling is handled using:

[https://docs.pytorch.org/docs/stable/generated/torch.Tensor.resize\\_.html](https://docs.pytorch.org/docs/stable/generated/torch.Tensor.resize_.html).

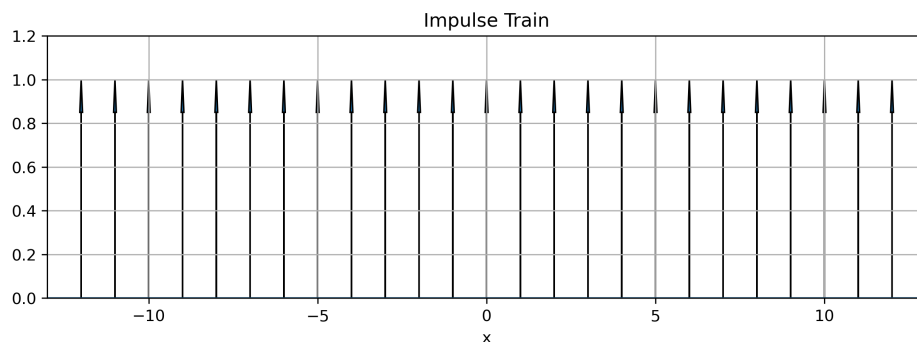
**1(a)** In 1-D, we define the impulse train using

$$s(x, T) = \sum_{n=-\infty}^{\infty} \delta(x - nT).$$

Fix the sampling period to  $T = T_0$ . Then, sketch the impulse train locations using an arrow to represent each  $\delta(\cdot)$ . Clearly indicate the locations of the samples.

**Solution.** Impulses occur at  $x = nT_0$  for  $n \in \mathbb{Z}$ , i.e.,

$$\dots, -2T_0, -T_0, 0, T_0, 2T_0, \dots$$



**1(b)** Expanding the sampling in multiple dimensions is simply based on:

$$S(x, y) = s(x, T_1)s(y, T_2).$$

Write down the final expression for  $S(x, y)$ . Fix the values for  $T_1$  and  $T_2$ . Sketch the impulse train locations in 2-D similarly to those in 1(a). Clearly indicate the locations of the samples. Also, clearly indicate how separability was used to derive the plot.

**Solution.** Using separability,

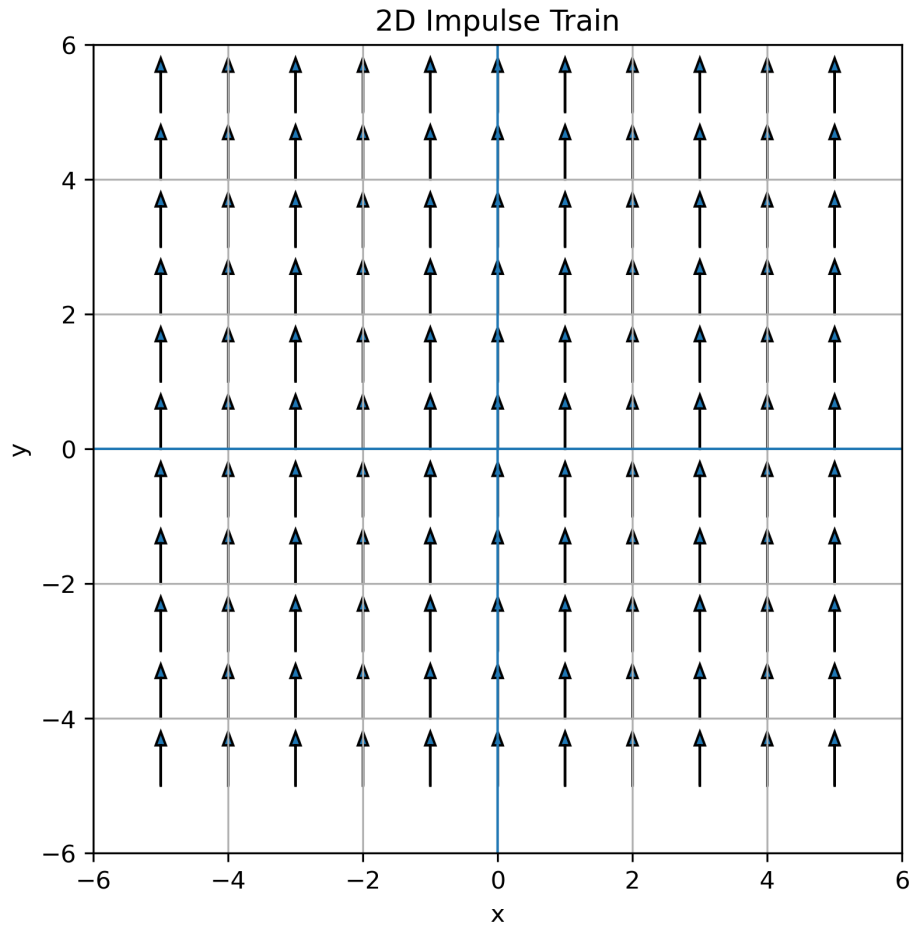
$$S(x, y) = s(x, T_1)s(y, T_2) = \left( \sum_{m=-\infty}^{\infty} \delta(x - mT_1) \right) \left( \sum_{n=-\infty}^{\infty} \delta(y - nT_2) \right),$$

so

$$S(x, y) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \delta(x - mT_1)\delta(y - nT_2).$$

Thus, the impulses are located at

$$(x, y) = (mT_1, nT_2), \quad m, n \in \mathbb{Z}.$$



**1(c)** In 1-D, we have the Fourier transform for 1-D given as:

$$\hat{s}(\xi, T) = \frac{1}{T} \sum_{n=-\infty}^{\infty} \delta\left(\xi - \frac{n}{T}\right)$$

where  $s(\cdot)$  is given in 1(a). Sketch the impulse train locations in the 1-D Fourier transform. Clearly indicate the locations of the samples.

**Solution.** The Fourier transform is

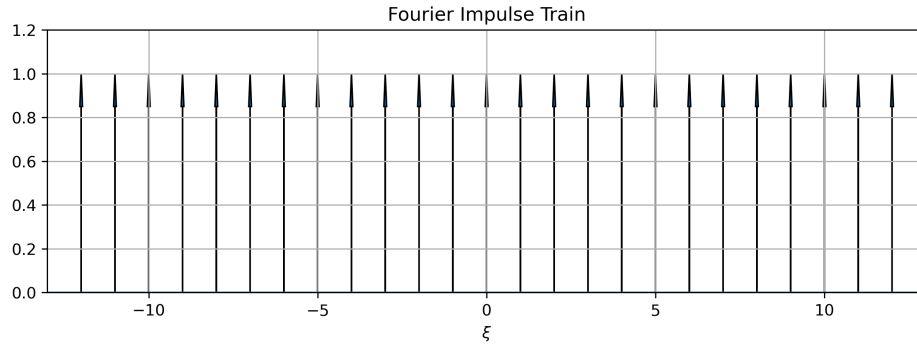
$$\hat{s}(\xi, T_0) = \frac{1}{T_0} \sum_{n=-\infty}^{\infty} \delta\left(\xi - \frac{n}{T_0}\right).$$

Thus, the impulses are located at

$$\xi = \frac{n}{T_0}, \quad n \in \mathbb{Z},$$

i.e.,

$$\dots, -\frac{2}{T_0}, -\frac{1}{T_0}, 0, \frac{1}{T_0}, \frac{2}{T_0}, \dots$$



**1(d)** Assuming that  $f(x) \in S(\mathbb{R}^2)$ , show that the Fourier transform is separable:

$$F(f(x)f(y)) = F_1(f(x))F_2(f(y))$$

and

$$\hat{f}(\xi_1, \xi_2) = F(f(x_1, x_2)) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) \exp(-2\pi i \xi \cdot x) dx_1 dx_2$$

where  $\xi \cdot x = \xi_1 x_1 + \xi_2 x_2$ , and  $F_1, F_2$  are defined over  $x_1, x_2$ .

**Solution.** Assuming  $f(x_1, x_2) = f_1(x_1)f_2(x_2)$ ,

$$\hat{f}(\xi_1, \xi_2) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_1(x_1)f_2(x_2)e^{-2\pi i(\xi_1 x_1 + \xi_2 x_2)} dx_1 dx_2.$$

Since

$$e^{-2\pi i(\xi_1 x_1 + \xi_2 x_2)} = e^{-2\pi i \xi_1 x_1} e^{-2\pi i \xi_2 x_2},$$

the integral separates as

$$\hat{f}(\xi_1, \xi_2) = \left( \int_{-\infty}^{\infty} f_1(x_1) e^{-2\pi i \xi_1 x_1} dx_1 \right) \left( \int_{-\infty}^{\infty} f_2(x_2) e^{-2\pi i \xi_2 x_2} dx_2 \right).$$

Thus,

$$F(f(x)f(y)) = F_1(f(x)) F_2(f(y)).$$

**1(e)** Apply the convolution theorem properties to show how sampling creates copies of the original spectrum of the image. To show this, note that sampling is viewed as multiplication

$$\text{Sampled } f = f(x, y) S(x, y).$$

Then, apply the property about multiplication in the time domain as convolution in the frequency domain and note that:

$$G(\xi_1, \xi_2) * \delta(\xi_1 - a, \xi_2 - b) = G(\xi_1 - a, \xi_2 - b).$$

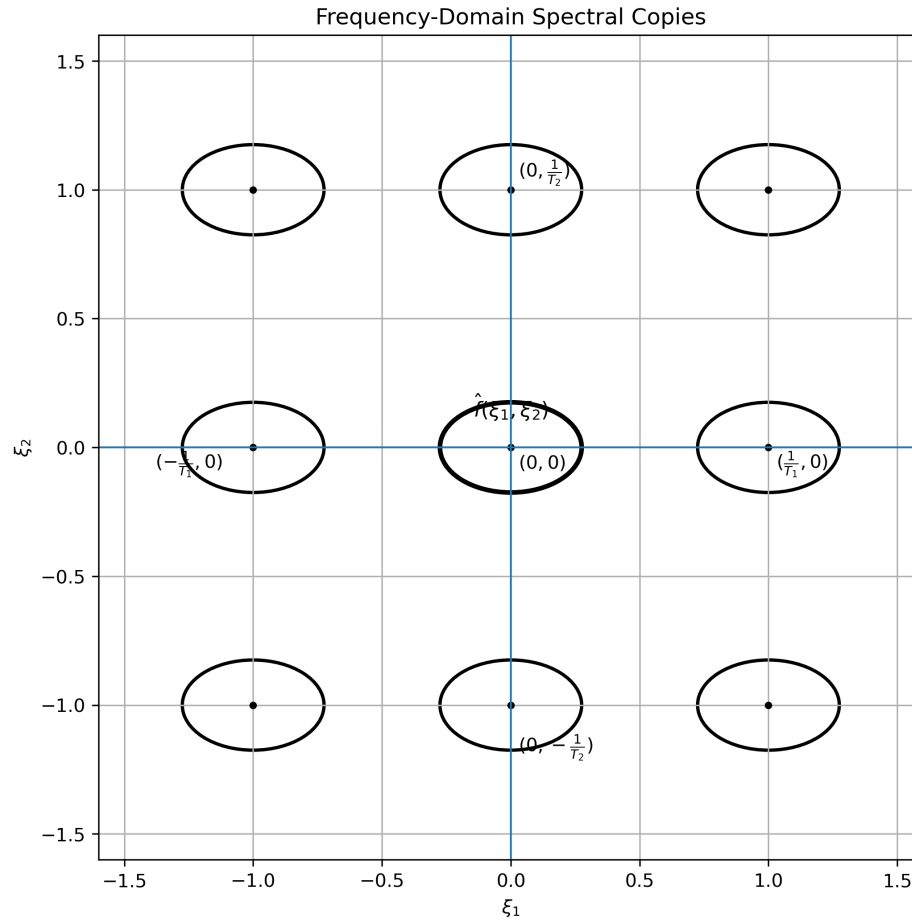
Sketch the copies of  $\hat{f}(\xi_1, \xi_2)$  and clearly indicate the locations of the spectra in 2-D.

**Solution.** Sampling produces shifted copies of the spectrum:

$$\hat{f}_s(\xi_1, \xi_2) = \frac{1}{T_1 T_2} \sum_{m, n \in \mathbb{Z}} \hat{f}\left(\xi_1 - \frac{m}{T_1}, \xi_2 - \frac{n}{T_2}\right).$$

Thus, copies of  $\hat{f}(\xi_1, \xi_2)$  appear at

$$\left( \frac{m}{T_1}, \frac{n}{T_2} \right).$$



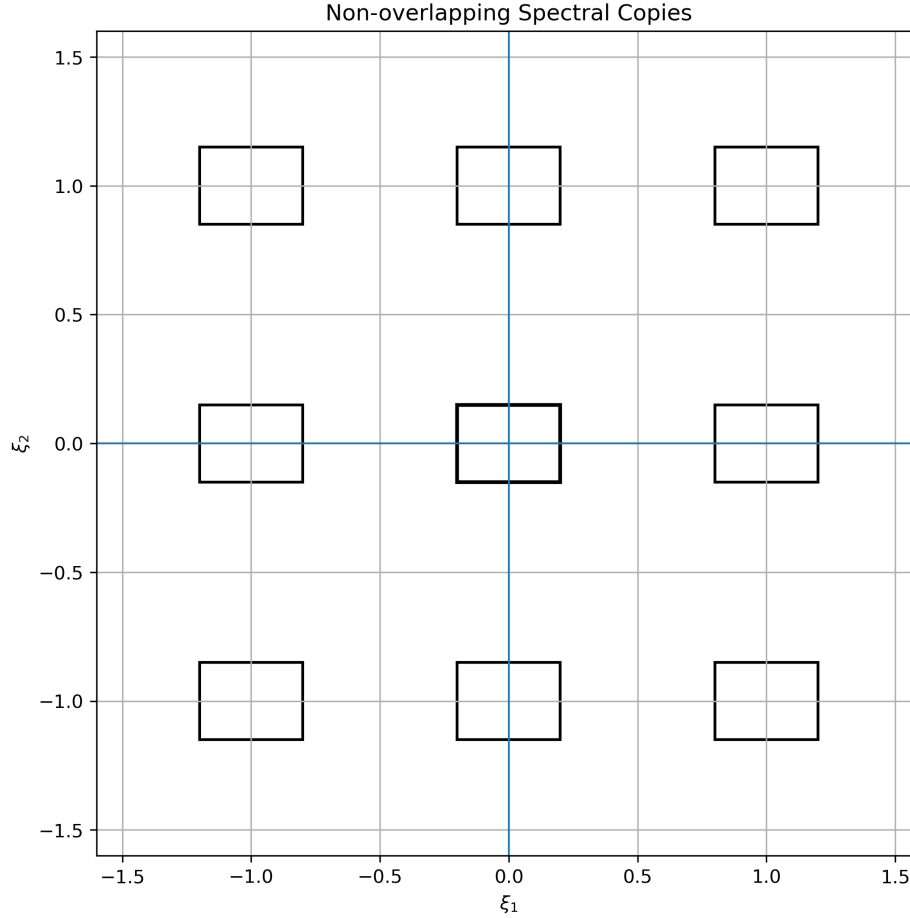
**1(f)** Derive the sampling theorem conditions that guarantee that the copies do not overlap. To do this, assume that the function is band-limited within a rectangular region of  $|\xi_1| \leq B_1$ , and  $|\xi_2| \leq B_2$  and then derive inequalities for  $T_1, T_2$  in terms of  $B_1, B_2$ . Sketch to show how the copies of the Fourier Transform do not overlap in the Frequency domain.

**Solution.** To avoid overlap of spectral copies, the spacing between replicas must exceed the bandwidth:

$$\frac{1}{T_1} > 2B_1, \quad \frac{1}{T_2} > 2B_2.$$

Equivalently,

$$T_1 < \frac{1}{2B_1}, \quad T_2 < \frac{1}{2B_2}.$$



**1(g)** Assume that an image is originally properly sampled as required in 1(f). Now, assume we use interpolation to increase the image size. Interpolation will not change the extent of the original (analog) bandwidth.

However, you can use the scaling theorem to see what will happen to the sampled extents of  $B_1, B_2$ . Consider the following function:

$$g(x_1, x_2) = f\left(\frac{x_1}{a}, \frac{x_2}{b}\right)$$

Compute the extents of the spectrum of  $g$  for  $a = 2$  and  $b = 3$ . The change in the spectrum due to interpolation will be visible in the digital spectrum measured using the FFT.

In this case, we are supposed to interpolate with sinc-functions, but the bi-cubic is a very good approximation. Indicate which of the three PyTorch functions given above you should use and how to set up the options for calling it.

**Solution.** For

$$g(x_1, x_2) = f\left(\frac{x_1}{2}, \frac{x_2}{3}\right),$$

the scaling theorem gives

$$\hat{g}(\xi_1, \xi_2) = 6 \hat{f}(2\xi_1, 3\xi_2).$$

Thus, the sampled spectrum appears stretched inside the FFT tile, while the analog bandwidth remains unchanged. For interpolation in PyTorch, the correct choice is `torch.nn.functional.interpolate`, for example

```
import torch.nn.functional as F
y = F.interpolate(x, scale_factor=(2,3), mode='bicubic')
```

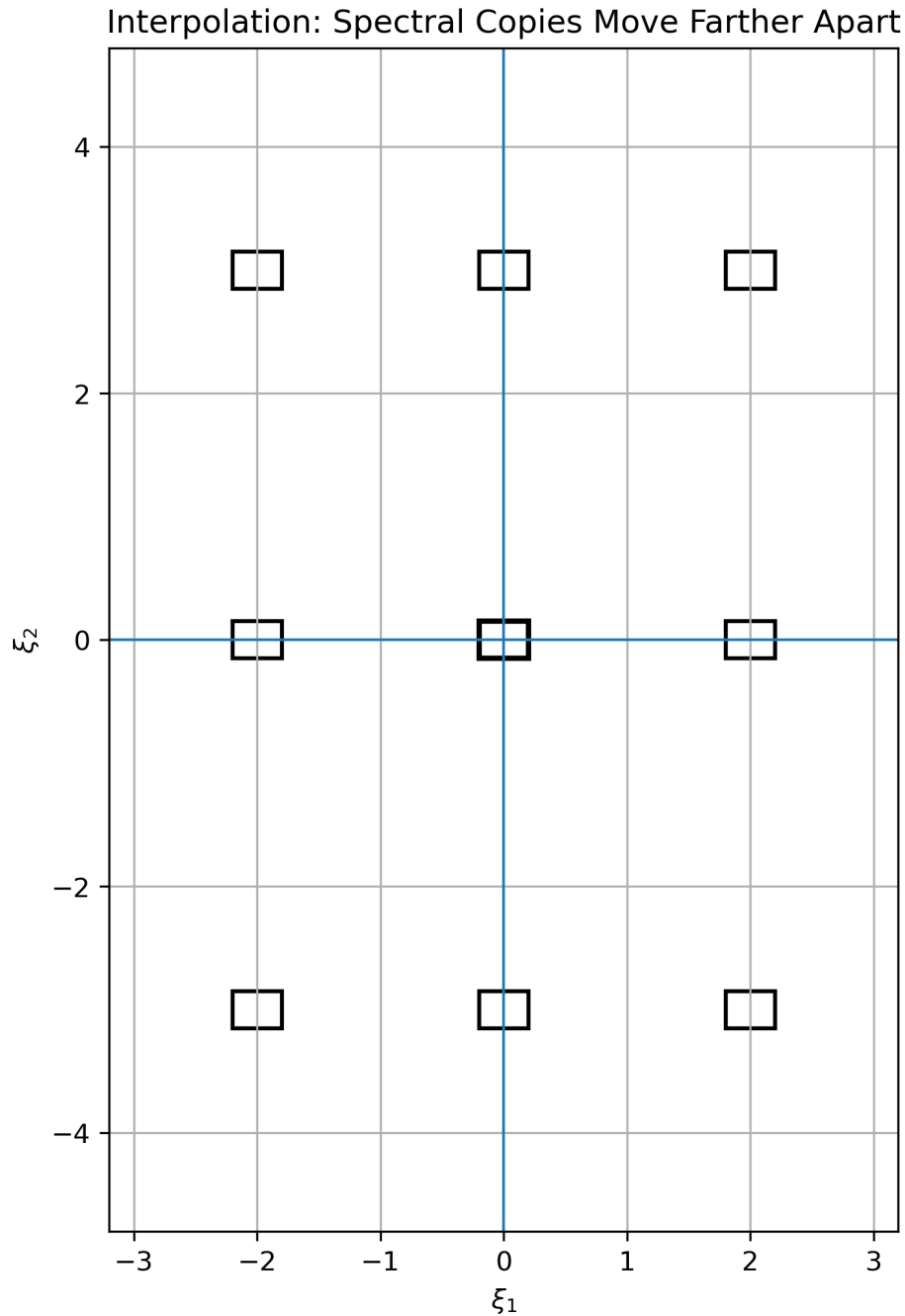
The function `torch.Tensor.resize_()` should not be used because it does not perform proper interpolation.

**1(h)** Will the copies overlap for interpolation? Demonstrate your answer using sketches. Remember that changing the sampling period does not change the original (analog) bandwidths. However, it changes how they appear within a “single tile” in the frequency domain. Thus, use the original  $B_1, B_2$  bandwidths and simply change the spacing here.

**Solution.** Interpolation reduces the sampling periods to  $T_1/2$  and  $T_2/3$ . Thus the spacing between spectral copies becomes

$$\frac{1}{T_1/2} = \frac{2}{T_1}, \quad \frac{1}{T_2/3} = \frac{3}{T_2}.$$

Since the bandwidth remains  $(B_1, B_2)$ , the spectral copies move farther apart and do not overlap.



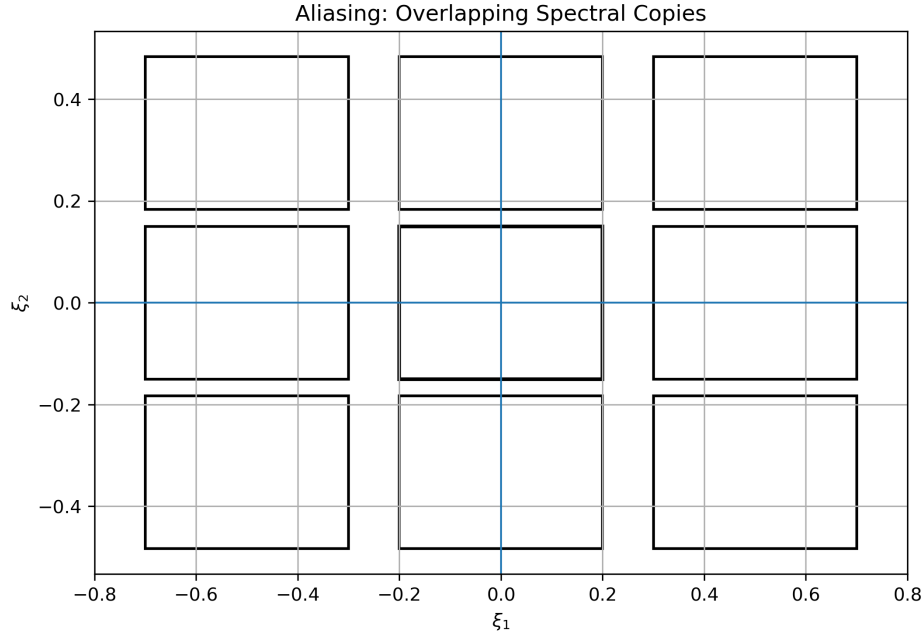
**1(i)** Repeat 1(h) for sampling at higher-periods:  $2T_1$  and  $3T_2$ . Show that the Fourier spectral copies will overlap in this case!

**Solution.** For sampling at  $2T_1$  and  $3T_2$ , the spacing between spectral copies becomes

$$\frac{1}{2T_1}, \quad \frac{1}{3T_2}.$$

Since the bandwidth remains  $(B_1, B_2)$ , the copies move closer together and overlap, producing aliasing.





**1(j)** In PyTorch, resampling is carried out using an array. A digital filter is required here to avoid sampling artifacts. Interpolation to a larger array size is much less of a problem. It can be implemented as outlined earlier.

Reducing the size of a digital image is a much more challenging problem, as explained in 1(i). This is handled in two steps.

The system requires two steps. First, apply the digital filter to reduce the digital spectrum of the original samples. Second, resample the image at the new sampling rate.

In digital form, the FFT can be used to compute the discrete Spectrum. If we are doubling the sampling periods, we must reduce the spectrum to half of the full digital spectrum in each dimension. There is a simple inverse relationship here. If the sampling period is increased to  $aT$  with  $a > 1$ , we need to reduce the original spectrum by a factor of  $a$  in that dimension.

For the example in 1(i), sketch the discrete-Spectrum (FFT-sampled) passbands that meet the requirements. Note that you need to remove the part of the spectrum that overlaps. This is accomplished by applying a digital filter with the indicated passband. Then clearly outline the parts of the spectrum that have been completely eliminated.

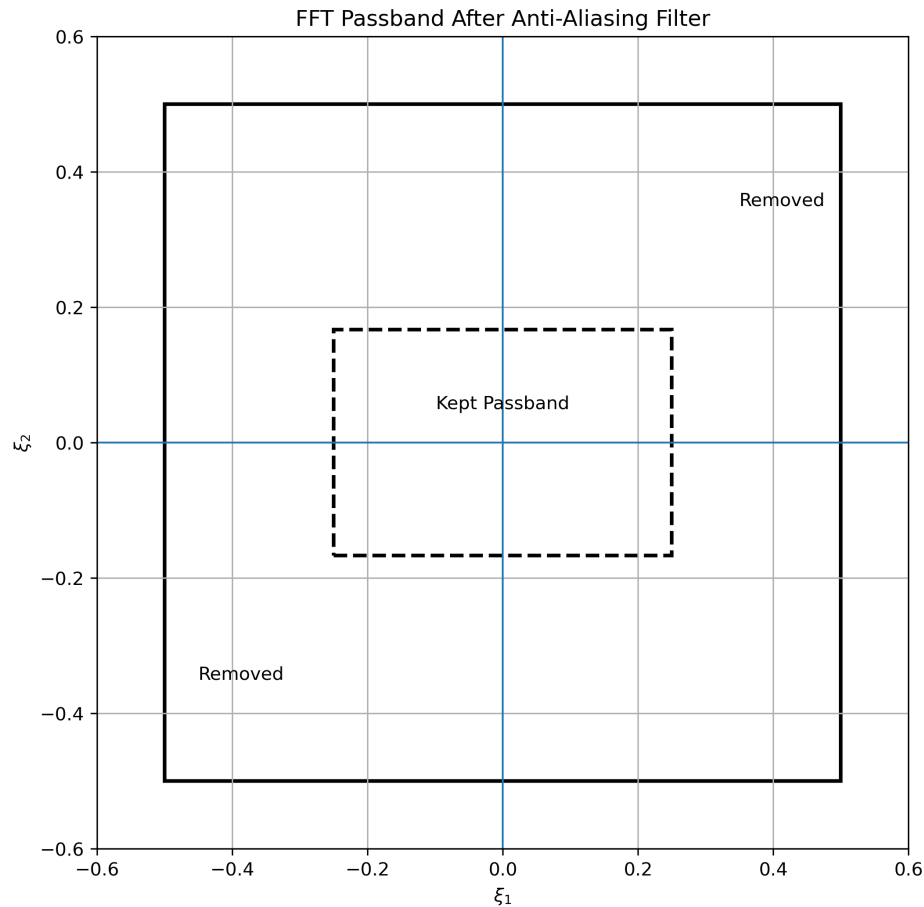
Explain how the eliminated spectra can create problems for further processing.

The only good option is to train the neural network on the original input image. Training on interpolated images should not produce problems. Resizing to much smaller image sizes can create many problems. Show that significant parts of the spectrum can be lost when training with much smaller images.

**Solution.** When the sampling periods increase to  $2T_1$  and  $3T_2$ , the spectral copies move closer together and overlap. To avoid aliasing, a digital low-pass filter must be applied before resampling. In the FFT domain, this means keeping only a smaller centered passband. Since the sampling period doubles in the  $\xi_1$  direction and triples in the  $\xi_2$  direction, the passband must shrink by factors of 2 and 3, respectively. Thus, only the central region of the spectrum is kept and the outer frequencies are removed.

The eliminated high-frequency components correspond to fine image details such as edges and

textures. Once removed, this information cannot be recovered, which may degrade later neural-network processing.



**1(k)** How would you implement image resampling in PyTorch for reducing the size of the input image? Discuss your options.

**Solution.** In PyTorch, image resampling for reducing the input size should be done by filtering first and then resizing in order to avoid aliasing. When the image is downsampled, high-frequency components can overlap in the spectrum, so a low-pass filter should remove those components before resampling.

In practice, this can be done using interpolation functions such as `torch.nn.functional.interpolate` or `torchvision.transforms.Resize`, which perform proper image resizing. For example:

```
import torch.nn.functional as F

y = F.interpolate(x, size=(H_new, W_new), mode='bicubic', antialias=True)
```

These methods perform interpolation and can apply anti-aliasing when reducing image size. The function `torch.Tensor.resize_()` should not be used because it only changes the tensor shape and does not perform proper image resampling.

```

# -----
# Homework 4 - Plot Generator
# Problems: 1(a), 1(b), 1(c), 1(e), 1(f), 1(h), 1(i), 1(j)
# -----

import os
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse, Rectangle

os.makedirs("plots", exist_ok=True)

# -----
# Shared parameters
# -----

T0 = 1.0
T1 = 1.0
T2 = 1.0

B1 = 0.2
B2 = 0.15

# -----
# Helper functions
# -----

def draw_1d_impulses(x_vals, xlabel, title, filename):
    plt.figure(figsize=(10, 3))
    for x in x_vals:
        plt.arrow(
            x, 0, 0, 1,
            head_width=0.08,
            head_length=0.15,
            length_includes_head=True
        )

    plt.axhline(0, linewidth=1)
    plt.xlim(min(x_vals) - 1, max(x_vals) + 1)
    plt.ylim(0, 1.2)
    plt.xlabel(xlabel)
    plt.title(title)
    plt.grid(True)
    plt.savefig(filename, dpi=300, bbox_inches="tight")
    plt.close()

def draw_2d_impulse_lattice(T1, T2, filename):
    m_vals = np.arange(-5, 6)
    n_vals = np.arange(-5, 6)

    fig, ax = plt.subplots(figsize=(6, 6))

    for m in m_vals:

```

```

for n in n_vals:
    x = m * T1
    y = n * T2
    ax.arrow(
        x, y, 0, 0.8,
        head_width=0.15,
        head_length=0.2,
        length_includes_head=True
    )

    ax.axhline(0, linewidth=1)
    ax.axvline(0, linewidth=1)
    ax.set_xlim(-6, 6)
    ax.set_ylim(-6, 6)
    ax.set_aspect("equal")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title("2D Impulse Train")
    ax.grid(True)

plt.savefig(filename, dpi=300, bbox_inches="tight")
plt.close()

def draw_spectral_copies_ellipses(dx, dy, filename):
    fig, ax = plt.subplots(figsize=(8, 8))
    spec_width = 0.55
    spec_height = 0.35

    for m in [-1, 0, 1]:
        for n in [-1, 0, 1]:
            cx = m * dx
            cy = n * dy
            ax.add_patch(Ellipse(
                (cx, cy),
                width=spec_width,
                height=spec_height,
                fill=False,
                linewidth=2.5 if (m, n) == (0, 0) else 1.8
            ))
    ax.plot(cx, cy, "k.", markersize=6)

    ax.axhline(0, linewidth=1)
    ax.axvline(0, linewidth=1)

    ax.text(0.03, -0.08, r'$(0,0)$', fontsize=10)
    ax.text(dx + 0.03, -0.08, r'$({T_1},0)$', fontsize=10)
    ax.text(-dx - 0.38, -0.08, r'$(-{T_1},0)$', fontsize=10)
    ax.text(0.03, dy + 0.05, r'$(0,{T_2})$', fontsize=10)
    ax.text(0.03, -dy - 0.18, r'$(0,-{T_2})$', fontsize=10)
    ax.text(0, 0.12, r'$\hat{f}(\xi_1,\xi_2)$', ha='center', fontsize=11)

    ax.set_xlim(-1.6 * dx, 1.6 * dx)
    ax.set_ylim(-1.6 * dy, 1.6 * dy)

```

```

ax.set_aspect("equal")
ax.set_xlabel(r'$\xi_1$')
ax.set_ylabel(r'$\xi_2$')
ax.set_title("Frequency-Domain Spectral Copies")
ax.grid(True)

plt.savefig(filename, dpi=300, bbox_inches="tight")
plt.close()

def draw_spectral_rectangles(dx, dy, B1, B2, title, filename):
    fig, ax = plt.subplots(figsize=(8, 8))

    for m in [-1, 0, 1]:
        for n in [-1, 0, 1]:
            cx = m * dx
            cy = n * dy
            ax.add_patch(Rectangle(
                (cx - B1, cy - B2),
                2 * B1,
                2 * B2,
                fill=False,
                linewidth=2 if (m, n) == (0, 0) else 1.5
            ))

    ax.axhline(0, linewidth=1)
    ax.axvline(0, linewidth=1)
    ax.set_xlim(-1.6 * dx, 1.6 * dx)
    ax.set_ylim(-1.6 * dy, 1.6 * dy)
    ax.set_aspect("equal")
    ax.set_xlabel(r'$\xi_1$')
    ax.set_ylabel(r'$\xi_2$')
    ax.set_title(title)
    ax.grid(True)

    plt.savefig(filename, dpi=300, bbox_inches="tight")
    plt.close()

def draw_filter_passband(filename):
    fig, ax = plt.subplots(figsize=(8, 8))

    orig_width = 1.0
    orig_height = 1.0
    pass_width = orig_width / 2
    pass_height = orig_height / 3

    outer = Rectangle(
        (-orig_width / 2, -orig_height / 2),
        orig_width,
        orig_height,
        fill=False,
        linewidth=2
    )

```

```

ax.add_patch(outer)

inner = Rectangle(
    (-pass_width / 2, -pass_height / 2),
    pass_width,
    pass_height,
    fill=False,
    linewidth=2,
    linestyle="--"
)
ax.add_patch(inner)

ax.text(0, 0.05, "Kept Passband", ha="center")
ax.text(0.35, 0.35, "Removed", fontsize=10)
ax.text(-0.45, -0.35, "Removed", fontsize=10)

ax.axhline(0, linewidth=1)
ax.axvline(0, linewidth=1)
ax.set_xlim(-0.6, 0.6)
ax.set_ylim(-0.6, 0.6)
ax.set_aspect("equal")
ax.set_xlabel(r'$\xi_1$')
ax.set_ylabel(r'$\xi_2$')
ax.set_title("FFT Passband After Anti-Aliasing Filter")
ax.grid(True)

plt.savefig(filename, dpi=300, bbox_inches="tight")
plt.close()

# -----
# Problem 1(a): Impulse Train
# -----

x_samples = np.arange(-12, 13) * T0
draw_1d_impulses(
    x_vals=x_samples,
    xlabel="x",
    title="Impulse Train",
    filename="plots/problem_1a_impulse_train.png"
)

# -----
# Problem 1(b): 2D Impulse Train
# -----

draw_2d_impulse_lattice(
    T1=T1,
    T2=T2,
    filename="plots/problem_1b_impulse_lattice.png"
)

# -----
# Problem 1(c): Fourier-domain Impulse Train
# -----

```

```

xi_samples = np.arange(-12, 13) / T0
draw_1d_impulses(
x_vals=xi_samples,
xlabel=r'$\xi$',
title="Fourier_Impulse_Train",
filename="plots/problem_1c_fourier_impulses.png"
)

# -----
# Problem 1(e): Frequency-Domain Spectral Copies
# -----

draw_spectral_copies_ellipses(
dx=1 / T1,
dy=1 / T2,
filename="plots/problem_1e_spectral_copies.png"
)

# -----
# Problem 1(f): Non-overlapping spectral copies
# -----

draw_spectral_rectangles(
dx=1 / T1,
dy=1 / T2,
B1=B1,
B2=B2,
title="Non-overlapping_Spectral_Copies",
filename="plots/problem_1f_no_overlap.png"
)

# -----
# Problem 1(h): Interpolation
# -----

draw_spectral_rectangles(
dx=2 / T1,
dy=3 / T2,
B1=B1,
B2=B2,
title="Interpolation:_Spectral_Copies_Move_Farther_Apart",
filename="plots/problem_1h_interpolation_spacing.png"
)

# -----
# Problem 1(i): Downsampling / Aliasing
# -----

draw_spectral_rectangles(
dx=1 / (2 * T1),
dy=1 / (3 * T2),
B1=B1,
B2=B2,

```

```
title="Aliasing: Overlapping Spectral Copies",  
filename="plots/problem_1i_aliasing.png"  
)
```

```
# -----  
# Problem 1(j): Digital filtering before downsampling  
# -----
```

```
draw_filter_passband(  
filename="plots/problem_1j_filter_passband.png"  
)
```